



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 8
по курсу «Языки и методы программирования»
«Разработка шаблона класса»

Студент группы ИУ9-21Б Белянин Г. И.

Преподаватель Посевин Д. П.

Москва 2026

1 Цель

Целью данной работы является изучение шаблонов классов языка C++.

2 Персональный вариант

`FenwickTree<T,N>` – дерево Фенвика для последовательности длины N с элементами типа T , имеющее операцию вычисления исключающего ИЛИ элементов на заданном отрезке и операцию изменения указанного элемента. Дерево должно быть реализовано через массив размера N . В `FenwickTree<bool, N>` каждому элементу должен соответствовать 1 бит.

3 Практическая реализация

Класс FenwickTree

```
#ifndef FENWICK_TREE_H
#define FENWICK_TREE_H

#include <iostream>
#include <vector>

template <typename T, int N>
class FenwickTree {
private:
    T tree[N + 1];
    T data[N];

public:
    FenwickTree() {
        for (int i = 0; i <= N; ++i) tree[i] = T();
        for (int i = 0; i < N; ++i) data[i] = T();
    }

    void update(int idx, T val) {
        if (idx < 0 || idx >= N) return;
        T delta = data[idx] ^ val;
        data[idx] = val;
        for (int i = idx + 1; i <= N; i += i & -i) {
            tree[i] ^= delta;
        }
    }

    T queryRange(int l, int r) const {
        if (l < 0 || r >= N || l > r) return T();
        auto prefix = [&](int idx) {
            T res = T();
            for (int i = idx + 1; i > 0; i -= i & -i) res ^= tree[i];
            return res;
        };
        return (l == 0) ? prefix(r) : (prefix(r) ^ prefix(l - 1));
    }
};

template <int N>
class FenwickTree<bool, N> {
private:
    unsigned char tree[(N + 1 + 7) / 8] = {0};
    unsigned char data[(N + 7) / 8] = {0};
};
```

```

void setBit(unsigned char* arr, int bitIdx, bool val) {
    if (val) arr[bitIdx >> 3] |= (1 << (bitIdx & 7));
    else arr[bitIdx >> 3] &= ~(1 << (bitIdx & 7));
}

bool getBit(const unsigned char* arr, int bitIdx) const {
    return (arr[bitIdx >> 3] >> (bitIdx & 7)) & 1;
}

public:
void update(int idx, bool val) {
    if (idx < 0 || idx >= N) return;
    bool oldVal = getBit(data, idx);
    bool delta = oldVal ^ val;
    setBit(data, idx, val);
    if (delta) {
        for (int i = idx + 1; i <= N; i += i & -i) {
            setBit(tree, i, getBit(tree, i) ^ true);
        }
    }
}

bool queryRange(int l, int r) const {
    if (l < 0 || r >= N || l > r) return false;
    auto prefix = [&](int idx) {
        bool res = false;
        for (int i = idx + 1; i > 0; i -= i & -i) res ^= getBit(tree, i);
        return res;
    };
    return (l == 0) ? prefix(r) : (prefix(r) ^ prefix(l - 1));
}
};

#endif

```

Main

```

#include <iostream>
#include "FenwickTree.h"

int main() {
    FenwickTree<int, 5> ft_int;
    ft_int.update(0, 1);
    ft_int.update(1, 2);
    ft_int.update(2, 4);
    std::cout << "Int XOR(0, 2): " << ft_int.queryRange(0, 2) << std::endl;
}

```

```

FenwickTree<bool, 10> ft_bool;
ft_bool.update(0, true);
ft_bool.update(1, true);
ft_bool.update(2, false);
std::cout << "Bool XOR(0, 2): " << std::boolalpha << ft_bool.queryRange
    (0, 2) << std::endl;

return 0;
}

```

4 Результаты

В результате работы программы получился следующий вывод:

```

Int XOR(0, 2): 7
Bool XOR(0, 2): false

```

5 Выводы

В ходе лабораторной работы были изучены и применены на практике механизмы шаблонного программирования в C++. Реализован параметризованный класс `FenwickTree<T, N>` для эффективного выполнения запросов обновления элемента и вычисления префиксного XOR (исключающего ИЛИ) на отрезке за логарифмическое время $O(\log N)$. Особое внимание уделено специализации шаблонов: разработана специализация `FenwickTree<bool, N>`, использующая битовую упаковку данных. Это позволило сократить потребление памяти в 8–32 раза (в зависимости от размера типа `int/char`) по сравнению с базовой реализацией, продемонстрировав преимущества шаблонов для создания типобезопасных и ресурсоэффективных структур данных. Работа подтвердила освоение навыков обобщенного программирования, включая работу с параметрами шаблона-типа (`T`) и параметрами-значениями (`N`), а также понимание особенностей битовых операций при работе с булевыми данными.